Taller Práctico: Árboles de Regresión y Clasificación en Python *

Estadística II *Universidad Nacional de Colombia, Sede Medellín*

Este documento corresponde al primer taller práctico sobre árboles e interpretabilidad del curso de **Estadística II** de la *Universidad Nacional de Colombia*, Sede Medellín, en el período 2026-02. El enfoque de este taller se centra en la construcción de un árbol por división binaria recursiva: cómo se elige cada corte reduciendo la suma de cuadrados de los residuales (RSS) o el índice de Gini, cómo se leen sus reglas para predecir una observación nueva y cómo se evalúa el árbol en datos que no participaron en su ajuste. **Monitor:** *Tomás Rodríguez Taborda.*

Keywords: árboles de regresión, árboles de clasificación, división binaria recursiva, índice de Gini, matriz de confusión

---

Información general

Con el propósito de profundizar en los conceptos de árboles de regresión y de clasificación vistos en clase, se propone afrontar este taller en dos partes: una de teoría básica y otra práctica.

Toda la parte computacional se desarrolla en Python, con `numpy`, `pandas`, `matplotlib` y `scikit-learn`. Use el notebook `PrimerTaller.ipynb` de esta carpeta y acompañe el código con las interpretaciones solicitadas. La parte teórica conserva sus demostraciones y cálculos justificados.

Use Python 3.11 o posterior. Antes de comenzar, instale las dependencias desde una terminal ubicada en esta carpeta:

```
python -m pip install -r requirements.txt
```

Abra el notebook en Jupyter o en VS Code y seleccione el entorno de Python donde instaló las dependencias. Ejecute las celdas en orden.

Parte teórica

Dé respuesta a las preguntas formuladas a continuación con base en la teoría tratada en clase. **Provea una interpretación cuando sea necesario**.

1. Sea R una región (hoja) de un árbol de regresión con m observaciones de entrenamiento $y_1, \ldots, y_m$ y media $\bar{y}_R$.

    - (a) Demuestre que, para cualquier constante c, $\sum_{i=1}^{m}(y_i - c)^2 = \sum_{i=1}^{m}(y_i - \bar{y}_R)^2 + m\,(\bar{y}_R - c)^2$, y concluya que la predicción constante que minimiza el RSS de la región es $\bar{y}_R$.

*El material asociado a este taller puede encontrarse en el repositorio del curso, **(https://github.com/torodriguezt/Estadistica-II)**

- (b) Use el resultado anterior para demostrar que dividir una región en dos **nunca aumenta** el RSS de entrenamiento. ¿Qué implica esto a medida que aumenta la profundidad del árbol? ¿Ocurre lo mismo con el error de prueba?

2. Considere seis jugadores de béisbol con sus años en las Grandes Ligas (X_1), sus hits en la temporada anterior (X_2) y su salario (Y, en cientos de miles de dólares):

Jugador	A	B	C	D	E	F
Años (X_1)	5	11	2	8	1	4
Hits (X_2)	85	160	140	120	70	100
Salario (Y)	1	8	4	6	2	3

Se construye un árbol de regresión con **al menos dos jugadores en cada grupo**.

- (a) Liste los cortes admisibles de cada predictora (puntos medios entre valores consecutivos distintos) y calcule, para cada uno, el RSS de cada grupo y el RSS total. ¿Cuál es la primera división y cuánto reduce el RSS frente a predecir el salario medio para todos?

- (b) Repita la búsqueda dentro de cada rama, comparando de nuevo las dos predictoras. ¿Por qué una de las ramas no puede dividirse? Dibuje el árbol final, con el número de jugadores y la predicción de cada hoja, y la partición del plano (X_1, X_2).

- (c) Prediga el salario de un jugador con 3 años y 150 hits, y el de otro con 12 años y 60 hits. ¿Influyen los hits en la segunda predicción? ¿Puede un árbol predecir un salario mayor que el máximo del entrenamiento para un jugador con más años que todos los observados? **Explique.**

3. Considere un nodo de un árbol de clasificación con dos clases, *Alto* y *Bajo*, en el que p es la proporción de la clase *Alto*.

- (a) Muestre que el índice de Gini del nodo es $G = 1 - p^2 - (1-p)^2 = 2p(1-p)$. ¿Para qué valor de p es máximo y cuándo vale cero?

- (b) Un nodo con 40 jugadores *Alto* y 40 *Bajo* puede dividirse con el corte A, en los grupos $(30, 10)$ y $(10, 30)$, o con el corte B, en los grupos $(20, 40)$ y $(20, 0)$, donde cada pareja es (*Alto*, *Bajo*). Compare los dos cortes según la tasa de error de clasificación ponderada (prediciendo la clase mayoritaria de cada grupo) y según el Gini ponderado. ¿Por qué se prefiere el índice de Gini para construir el árbol?

- (c) Un nodo $(6, 62)$ se divide en las hojas $(1, 48)$ y $(5, 14)$. Verifique que ambas hojas predicen *Bajo* y que el número de errores de entrenamiento no cambia, pero el Gini ponderado sí disminuye. ¿Qué aporta entonces esta división?

4. Determine el valor de verdad de las siguientes afirmaciones. **Justifique en todos los casos.**

   - (a) La división binaria recursiva encuentra el árbol con menor RSS de entrenamiento entre todos los árboles con el mismo número de hojas.
   - (b) Para buscar el mejor corte de una predictora basta con evaluar los puntos medios entre sus valores consecutivos distintos, pues cualquier otro punto entre ellos deja las mismas observaciones a cada lado.
   - (c) Si una predictora no aparece en las reglas que recorre una observación hasta su hoja, cambiar su valor no modifica la predicción de esa observación.
   - (d) Si el MSE de prueba de un árbol es mucho mayor que su MSE de entrenamiento, el árbol está sobreajustado.
   - (e) Si un árbol predice salarios mayores para los jugadores con más años, un año adicional de experiencia causa un aumento del salario.

Ejercicio con datos reales

Considere el conjunto de datos `Hitters`, con el salario de 1987 y las estadísticas de desempeño de 322 jugadores de las Grandes Ligas de béisbol. Se carga en Python con `load_data("Hitters")` del paquete `ISLP`. **Se incluyen únicamente algunas variables**: *Salary*, el salario anual en miles de dólares; *Years*, los años en las Grandes Ligas, y *Hits*, los hits (batazos válidos) de la temporada de 1986.

El siguiente código prepara los datos y muestra las primeras cinco filas:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from ISLP import load_data
SEMILLA = 2026
hitters = (
    load_data("Hitters")
    .dropna(subset=["Salary"])
    .loc[:, ["Salary", "Years", "Hits"]]
    .reset_index(drop=True)
)
X = hitters[["Years", "Hits"]]
y = np.log(hitters["Salary"]).rename("log(Salary)")
X_ent, X_pru = X.iloc[:200], X.iloc[200:]
y_ent, y_pru = y.iloc[:200], y.iloc[200:]
assert len(hitters) == 263
assert len(X_ent) == 200 and len(X_pru) == 63
print(hitters.head())
```

Se eliminan los jugadores sin salario registrado (quedan 263), se usa $Y = \log(\textit{Salary})$ como respuesta y se toman las primeras 200 filas como entrenamiento y las 63 restantes como prueba, **sin aleatorizar ni cambiar el orden de las filas**. `np.log` calcula el logaritmo natural. Calcule el RSS, el MSE y el RMSE en esta escala; transforme las predicciones a miles de dólares solo cuando se solicite. **Dé respuesta a los siguientes planteamientos:**

1. Grafique $\log(\textit{Salary})$ frente a *Years* y frente a *Hits* en el entrenamiento, y **explique** por qué se modela el logaritmo del salario. Obtenga la predicción que se haría sin usar las predictoras y su RSS de entrenamiento.

2. **Programe una función** que, para una predictora numérica, evalúe todos los cortes admisibles (puntos medios entre valores consecutivos distintos que dejan al menos 10 jugadores a cada lado) y devuelva el RSS izquierdo, el derecho y el total de cada uno. Aplíquela a *Years* y a *Hits*, grafique el RSS total frente al punto de corte y reporte el número de cortes admisibles, el mejor corte de cada predictora y la primera división del árbol.

3. Ajuste con `DecisionTreeRegressor` un árbol de regresión con profundidad máxima 2, al menos 20 jugadores para intentar una división y 10 por hoja. Dibújelo con `plot_tree`, reporte la regla, el número de jugadores y la predicción de cada hoja, y represente la partición del plano (*Years*, *Hits*). **Describa las cuatro regiones**. ¿Por qué el corte en *Hits* es distinto en cada rama?

4. Para tres jugadores nuevos con (*Years*, *Hits*) iguales a $(3, 150)$, $(7, 90)$ y $(12, 160)$, siga las reglas del árbol hasta su hoja, obtenga la predicción y verifíquela con la función de predicción. Exprésela en miles de dólares mediante $e^{\widehat{y}}$ y **explique** por qué este valor no es el salario promedio de los jugadores de la hoja.

5. Compare el RSS, el MSE y el RMSE de entrenamiento y de prueba de la media de entrenamiento, del árbol del punto 3 y de una regresión lineal múltiple con *Years* y *Hits*. ¿Qué modelo predice mejor a los jugadores de prueba? **Relacione** el resultado con los gráficos del punto 1.

6. Para profundidades máximas de 1 a 6, obtenga el MSE de entrenamiento y el de validación cruzada de 10 particiones (calculado **solo con el entrenamiento**). Elija la profundidad que minimiza el MSE de validación cruzada; en caso de empate, prefiera la menor. Después calcule el MSE de prueba de cada profundidad y grafique las tres curvas, sin cambiar la elección anterior. **Justifique** por qué se elige con validación cruzada y no con el conjunto de prueba. Relacione la curva de entrenamiento con el punto 1(b) de la parte teórica.

7. Defina la clase *Alto* si $\textit{Salary} \geq 500$ y *Bajo* en otro caso, y reporte su proporción en el entrenamiento y en la prueba. Adapte la función del punto 2 al Gini ponderado para hallar la primera división, y ajuste con `DecisionTreeClassifier` un árbol de clasificación con el índice de Gini, profundidad máxima 2 y los mismos tamaños mínimos del punto 3. Reporte las proporciones y la clase predicha de cada hoja, identifique una división cuyas dos hojas predicen la misma clase y **relaciónela** con el punto 3(c) de la

parte teórica. Para los jugadores del punto 4, obtenga $\widehat{P}(Alto)$ con `predict_proba` y la clase predicha con `predict`.

8. Con *Alto* como clase positiva, obtenga la matriz de confusión de prueba, la tasa de clasificación correcta (TCC), la tasa de error, la sensibilidad y la especificidad del árbol del punto 7, de un árbol de profundidad máxima 1 y de una regresión logística con *Years* y *Hits* (umbral 0.5). **Discuta** qué revela la matriz de confusión que la TCC por sí sola no muestra.